

Helix: A Cloud-Native Multi-Tenant Control Plane for Synchronous Agentic Workspaces

Project Title

Helix — A Cloud-Native Multi-Tenant Control Plane for Synchronous Agentic Workspaces

One-Line Pitch

Helix turns isolated AI chat sessions into shared, branchable, monitored team workspaces — so engineering teams can collaborate with AI the same way they collaborate on code.

Problem Statement

Modern AI interfaces like Claude and ChatGPT are designed for one person and one conversation. When teams try to use AI collaboratively on complex engineering or research tasks, three structural failures occur:

- 1. Siloed Context** Every teammate starts from scratch in their own tab. There is no shared session, no live visibility into what a colleague is prompting, and no accumulated team memory. Redundant queries waste both time and API budget.
- 2. Linear Chat Deadlocks** Standard chat threads cannot branch. When a conversation reaches a breakthrough or a fork in reasoning, there is no way to clone that state and

explore two directions simultaneously — doing so either breaks the live session or loses one thread entirely.

3. Unmonitored Agent Autonomy Long-running recursive AI agents (performing file edits, API calls, cloud operations) operate as black boxes. There is no mid-loop visibility, no kill switch, and no cost guardrail — meaning a runaway agent can cause irreversible damage before anyone notices.

Helix solves all three by introducing a shared workspace layer that sits above any LLM API, making AI sessions a team resource with live co-presence, a branchable context tree, and a real-time agent monitoring dashboard.

Target Users

User	Pain Being Solved
Engineering teams (3-15 devs)	Rebuilding context every session; no shared AI history
Research groups	No versioning or reproducibility across AI experiments
Agentic workflow operators	Zero visibility into running agents; risk of runaway loops
Educators & cohorts	Cannot observe, guide, or fork a student's AI session

Modules

M1 — Auth & Workspace Registry

User accounts, workspace creation, invite-link onboarding, and role assignment (Owner

/ Collaborator / Observer). JWT-based authentication. Every subsequent module scopes to a workspace tenant.

M2 — Shared Context Engine

Maintains a single, authoritative conversation state per workspace. Routes each user's prompt into the shared thread in sequence. Manages context window budget — triggers summarisation when the limit approaches to keep costs bounded.

M3 — Real-Time Sync Layer

WebSocket rooms (one per workspace) broadcast prompt events, streaming token responses, and presence signals to all connected clients. Target latency under 200ms. Built on Socket.io with Redis pub/sub for multi-instance scaling.

M4 — Session Fork Engine

Snapshots the current conversation state as a node in a persistent tree. Creates a child branch that inherits full parent context but evolves independently. Both branches continue in parallel. A sidebar UI visualises the branch tree — think Git history, but for conversations.

M5 — Agent Execution Runtime

Wraps Claude's Tool Use API in a controlled execution loop (ReAct: reason → act → observe → repeat). Runs tool calls in a sandboxed environment. Emits a structured step event for every action taken — tool name, input, output, latency, cost.

M6 — Agent Monitor Dashboard

Live dashboard consuming M5 step events. Shows the full execution trace in real time. Provides a kill switch (pause or terminate the agent at any step), recursion depth guard, and a cost ticker with configurable threshold alerts.

M7 — Session History & Replay

Persists the full conversation tree to the database. Any team member can replay any session step by step, including forked branches. Exportable as JSON or Markdown for documentation or audit.

M8 — Prompt Permission Layer

Owner-controlled rules: restrict which tools an agent may call, require human approval before high-risk tool executions, and set per-role prompt injection limits. Implements a basic capability system on top of RBAC.

Specialisation Concepts

Agentic AI & the ReAct Loop

Helix implements a controlled agentic execution runtime using the ReAct (Reason-Act-Observe) pattern via Claude's Tool Use API. Agents plan, execute tools, observe outputs, and iterate — all under monitored, interruptible conditions. This is applied agentic AI engineering, not just prompt chaining.

Distributed Systems — Shared Mutable State

Multiple users writing to a single conversation context simultaneously is a classic distributed consistency problem. Helix uses serialised prompt queuing with sequence numbers to ensure every client sees a consistent, ordered conversation state — a simplified form of operational transformation.

Persistent Tree Data Structures

The session fork model is a persistent branching tree (structurally similar to Git's commit graph). Each fork creates a new node with a pointer to the parent snapshot. Structural sharing ensures forking is $O(1)$ — no deep copy of the full conversation history.

Multi-Tenancy & Resource Isolation

Each workspace is a fully isolated tenant. Row-level security in PostgreSQL and tenant-scoped Redis keyspaces prevent any context, agent log, or API credential from crossing workspace boundaries. This is production-grade multi-tenant architecture.

Pub/Sub Messaging

Redis pub/sub decouples the WebSocket event layer from the AI response stream. Multiple backend instances can subscribe to workspace events — the core primitive that enables horizontal scaling without sticky sessions.

Streaming I/O & Backpressure Management

Claude's API returns responses as a token stream. Helix fans that stream out to N simultaneous WebSocket clients. Backpressure handling (ensuring slow clients do not stall the stream) is a real systems engineering concern addressed in the sync layer.

Role-Based Access Control (RBAC)

Every action in the system (send prompt, fork session, kill agent, view logs, approve tool call) is a discrete permission. Roles map to permission sets via a policy table — not ad-hoc conditional logic — enabling clean audit trails and future extensibility.

Context Window Management

With multiple users contributing prompts, the shared context fills faster than in a solo session. Helix implements a sliding-window summarisation strategy using Claude itself to compress older turns while preserving key facts — applied NLP engineering within a cost constraint.

SDG Goals

SDG 4 — Quality Education

Helix enables a shared AI learning environment. An instructor can open a live session with students, fork individual threads to address specific misconceptions, and replay the session as a structured teaching artefact. It democratises access to collaborative AI-assisted education without requiring each participant to hold an individual subscription.

SDG 8 — Decent Work & Economic Growth

By eliminating redundant AI queries and pooling API spend across a team, Helix directly reduces operational cost for engineering and research teams. The agent monitor prevents runaway agents from generating unexpected cloud and API costs — a documented, real-world economic risk for teams adopting agentic AI.

SDG 9 — Industry, Innovation & Infrastructure

Helix advances the infrastructure layer for responsible, collaborative AI deployment. The agent monitoring runtime is a concrete engineering contribution to safe agentic systems — directly relevant to enterprise AI adoption and the broader AI safety research agenda.

SDG 10 — Reduced Inequalities

A shared workspace model allows under-resourced teams to pool a single API budget and receive collective benefit — rather than requiring individual subscriptions per team member. This levels access for small teams and institutions in lower-income regions.

SDG 17 — Partnerships for the Goals

Helix is, at its core, collaborative infrastructure. It makes cross-team and cross-institutional AI collaboration structurally possible. An open-source release would directly enable global research and educational partnerships around shared AI tooling.

Tech Stack Summary

Layer	Technology
Frontend	React + TypeScript, TailwindCSS, Vite
Backend	Node.js + Fastify, Socket.io
Database	PostgreSQL (persistent), Redis (ephemeral + pub/sub)
AI Layer	Anthropic Claude API (streaming + Tool Use)
Infrastructure	Docker + Docker Compose, GitHub Actions (CI/CD)
Deployment	Railway / Fly.io

8-Week Roadmap

Weeks	Milestone	Modules
1-2	Foundation	M1 (Auth), M2 (Shared Context), single-user Claude proxy
3-4	Real-time collaboration	M3 (WebSocket sync), multi-user prompt routing, presence
5	Session forking	M4 (Fork engine), branch tree UI
6-7	Agent layer	M5 (Agent runtime), M6 (Monitor dashboard)

Weeks	Milestone	Modules
8	Hardening & demo	M7 (Replay), M8 (Permissions), performance testing, demo video

What Makes This Novel

No existing tool provides all three of: shared live AI sessions, branchable conversation state, and real-time agentic oversight in a single platform. Helix is not a wrapper around Claude — it is a collaboration and safety layer that any LLM API can run under. The session fork primitive in particular has no direct equivalent in any current commercial AI product.

Team & Constraints

- **Team size:** 3
- **Duration:** 2 months
- **AI access:** Claude Pro (all members)
- **Suggested split:** Person A → backend & infra · Person B → frontend & UX · Person C → AI layer & agent runtime

Minimum viable demo: shared real-time session (M1-M3) + one live agent with kill switch (M5-M6). Everything else is depth, not breadth.

Helix — built for teams that think together.